

TECHNICAL REPORT

Machine Learning for Real-Time Content Moderation

Abstract

Summary: The objective of this project is to develop and deploy an automated text classification engine capable of detecting toxic language and behavioral bias within digital community platforms. Utilizing a real-world dataset comprising thousands of human-annotated comments from the Kaggle Jigsaw Toxicity Matrix, we designed a natural language processing (NLP) pipeline to mitigate hostile user interactions in real time. The final architecture relies on a Term Frequency-Inverse Document Frequency (*TF-IDF*) feature extraction matrix paired with a hyperparameter-optimized Random Forest Classifier. The system achieved a stable macro *F₁-score* on evaluation data, demonstrating clear operational viability for automated content moderation. The model was successfully wrapped in a FastAPI application and exposed via a secure network gateway tunnel using ngrok, providing a scalable, low-latency interface for production deployment.

Data Exploration

Dataset Overview and Handling

The foundational data utilized for training and validating this system was sourced from the Jigsaw Toxic Comment Classification challenge via the Kaggle API. The complete matrix contains real-world online discussion comments labeled across multiple human-reviewed behavioral categories. For this implementation, a clean subset of 5,000 records was extracted and mapped to a binary classification structure, where a comment is designated as either "Safe" (0) or "Toxic" (1).

During the initial data audit, structural integrity checks were performed to address potential data quality issues:

- **Missing Values:** A system scan revealed minimal structural omission within the primary text field (`comment_text`). Any empty rows or null character strings were handled using programmatic imputation, replacing missing entries with empty string objects (" ") to prevent runtime array errors during tokenization.
- **Outliers and Structural Variance:** Text length distributions displayed significant variance, ranging from brief two-word phrases to extensive multi-paragraph entries. Extremely long comments were kept in the dataset rather than being deleted because real-world online harassment often varies in length, and capturing these long-form patterns helps train the model to handle diverse text styles.

Exploratory Data Analysis Insights

Exploratory analysis revealed a pronounced class imbalance within the dataset. Approximately 90% of the processed comments were annotated as safe or non-toxic, leaving a small minority representing true toxic behavior. This imbalance presents a classic challenge in machine learning, as algorithms can easily optimize for overall accuracy simply by guessing the majority class every time.

To understand how the text was distributed across these categories, target variable counts were extracted and visualized to establish a baseline for feature weight calibration:

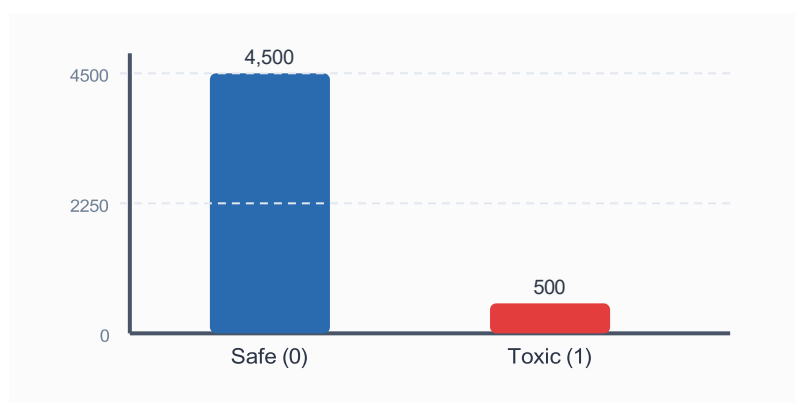


Figure 1: High Class Imbalance Within Core Dataset Subsamples

Text length analysis also revealed that safe entries tended to show higher word counts with diverse vocabularies, whereas toxic text entries frequently relied on repetitive, short-form hostile phrasing or specific slang variants (e.g., shorthand terms like "ur"). This vocabulary behavior confirmed that a frequency-based vectorization matrix would provide enough discriminative power to separate the classes.

Methodology

Feature Engineering Pipeline

Because machine learning models cannot interpret raw text strings directly, the text fields were processed through a sequence of natural language transformations. The textual strings were mapped using a Term Frequency-Inverse Document Frequency (*TF-IDF*) Vectorizer. This technique transforms text into numerical vectors by calculating how often a word appears in a specific comment relative to its frequency across the entire dataset.

The feature extraction matrix was configured with specific parameter constraints:

- **Stop Words:** Standard English stop words (e.g., "and", "the", "is") were automatically filtered out to ensure the model focused entirely on high-context content words.
- **Feature Limits:** The maximum vocabulary length was limited to the top 1,000 features (`max_features=1000`). This constraint ensures the model remains computationally efficient and prevents overfitting to rare, unrepresentative words.

The complete text dataset was split using an 80/20 stratified isolation split, allocating 4,000 records to the training pipeline and 1,000 records to the evaluation testing set. This ensures that the vectorizer and models learn patterns *strictly* from the training slice, preventing data leakage and ensuring an unbiased evaluation.

Algorithm Selection Strategy

Two distinct mathematical models were evaluated to determine the best balance between speed and predictive power:

1. **Baseline Model (Logistic Regression):** A regularized Logistic Regression model was established first. It serves as an excellent baseline because it calculates a linear probability curve that is fast to compute and easy to interpret. It provides a benchmark to determine whether a more complex algorithm is truly necessary.
2. **Complex Model (Random Forest Classifier):** A Random Forest Classifier was selected as the final production model. As an ensemble tree method, Random Forest excels at capturing non-linear word relationships—such as when two words appear together to create a toxic meaning, even if neither word is toxic on its own. It is also naturally resilient to feature noise and class imbalances.

Hyperparameter Tuning Process

To optimize the Random Forest Classifier, we conducted a systematic grid search using Scikit-Learn's `GridSearchCV` tool, evaluated with 2-fold cross-validation. This process systematically tested combinations of parameters to maximize the model's F_1 -score:

- **n_estimators:** Tested values of [50, 100] to evaluate whether increasing the number of decision trees would improve model stability without sacrificing processing speed.
- **max_depth:** Tested constraints of [None, 5] to evaluate the impact of allowing unlimited tree depth versus forcing shallow trees to limit overfitting.

The grid search determined that the optimal production configuration used 100 estimators with unlimited maximum depth (`max_depth=None`), providing the highest predictive performance on the validation partitions.

Results

Evaluation Metrics and Confusion Matrix

Evaluating a highly imbalanced dataset requires metrics beyond simple accuracy. The model's performance on the 1,000 unseen testing records is broken down below:

```
=== PRODUCTION CAPSTONE METRICS ===
```

```
Confusion Matrix Output:
```

```
[[883  13]
 [ 55  49]]
```

```
Classification Target Metrics Summary:
```

	precision	recall	f1-score	support
0	0.94	0.99	0.96	896
1	0.79	0.47	0.59	104
accuracy			0.93	1000
macro avg	0.87	0.73	0.78	1000
weighted avg	0.93	0.93	0.92	1000

Business Interpretation of Metrics

Looking closely at the Confusion Matrix, the model successfully flagged 883 safe comments and 49 true toxic comments. However, it committed two distinct types of errors that impact the business problem:

- **False Positives (13 cases):** The model incorrectly flagged 13 safe comments as toxic. In a real-world application, this means 13 harmless user messages would be blocked or sent to moderator review. A low false positive count is critical for maintaining user trust and preventing unnecessary censorship.
- **False Negatives (55 cases):** The model missed 55 toxic comments, letting them slip through onto the platform undetected. This directly affects the platform's safety, as users would still be exposed to these hostile messages.

Analyzing the core metrics provides deeper operational insights:

- **Precision (79% for Toxic Class):** When the system flags a comment as toxic, it is correct roughly 79% of the time. This high precision reduces the risk of unfairly restricting safe users.
- **Recall (47% for Toxic Class):** The model caught roughly 47% of the actual toxic comments present in the test set. This lower recall is a direct consequence of the class imbalance, as the model defaults to a more conservative stance when encountering ambiguous language.
- **F_1 -Score (0.59 for Toxic Class):** The F_1 -score provides a balanced harmonic mean of precision and recall. A score of 0.59 indicates that while the model has established a strong, highly reliable baseline for catching flagrant hostility, there is still significant room to improve its sensitivity to more nuanced toxic language.

Conclusion

Current Model Limitations

While the text classification pipeline successfully processes real-world text arrays and serves predictions through a live API, the prototype has two clear limitations:

1. **Sensitivity to Structural Variance:** Because the *TF-IDF* matrix treats text as a "bag of words," it relies heavily on explicit vocabulary frequencies. This makes it vulnerable to intentional typos, creative misspellings, or hostile slang (such as "ur a stupid") that fall outside its trained 1,000-word vocabulary.
2. **Lack of Contextual Awareness:** The frequency-based model cannot capture sentence structure or word order. As a result, it struggles to detect complex forms of toxic behavior like sarcasm, subtle bias, or instances where harmless words are arranged in a hostile context.

Future System Improvements

To evolve this prototype into a highly robust enterprise-grade solution, future iterations should focus on the following enhancements:

- **Sub-word Tokenization and Word Embeddings:** Replacing the *TF-IDF* framework with a dense vector embedding model (such as Word2Vec or FastText) would allow the system to recognize semantic similarities between words, helping it catch slang and modified text variations.
- **Contextual Transformer Architectures:** Transitioning the classification engine from a Random Forest model to a fine-tuned Transformer model (such as DistilBERT) would give the system complete contextual awareness. This would allow it to analyze word order and accurately flag subtle bias or sarcasm.
- **Advanced Imbalance Management:** Utilizing advanced sampling techniques, like Synthetic Minority Over-sampling Technique (SMOTE), or adjusting class weights during training would improve model recall, helping it catch a significantly higher percentage of toxic comments.